

Resumen ejecutivo

1. Resumen Ejecutivo y Valor de Negocio

En el entorno industrial de alta exigencia, la gestión manual de equipos de I+D (R&D) y los procesos basados en papel generan cuellos de botella logísticos y riesgos de pérdida de capital financiero. Para resolver esta problemática, se ha diseñado e implementado **ZF Halo**: un ecosistema de control patrimonial seguro, estructurado bajo una arquitectura de software robusta, cuya misión es garantizar la visibilidad absoluta del ciclo de vida de cada activo.

Toda la infraestructura de ZF Halo ha sido diseñada para proteger el activo más valioso de la organización: la información. La plataforma garantiza una **trazabilidad inmutable**. La dirección ya no tiene que estimar la ubicación de los equipos, gracias a la implementación de un *dashboard* interactivo en tiempo real que permite visualizar indicadores clave de rendimiento (KPIs) de disponibilidad, préstamos vencidos y el uso desglosado a través de todas las disciplinas de negocio.

El valor de negocio y tecnológico de esta solución se fundamenta en los siguientes pilares:

- **Eficiencia Operativa y Accesibilidad (PWA):** Construido bajo un enfoque *Mobile-First* y como Aplicación Web Progresiva (PWA), el sistema ofrece una experiencia nativa sin consumir almacenamiento local. Garantiza que cualquier ingeniero pueda solicitar un activo en un máximo de tres interacciones, mientras que el Modo Kiosco reduce el tiempo de procesamiento de salida en caseta a menos de 2 segundos.
- **Seguridad de Grado Empresarial:** La arquitectura implementa un estricto Control de Acceso Basado en Roles (RBAC), fortificado por JSON Web Tokens (JWT), validación Captcha y Autenticación de Dos Factores (2FA). Además, el sistema mitiga activamente los riesgos del OWASP Top 10 mediante validación y sanitización estricta de datos.
- **Cumplimiento y Escalación Automatizada:** El motor de base de datos orquesta el viaje de cada equipo, ejecutando disparadores de cumplimiento (*compliance triggers*) que envían recordatorios automáticos 48 y 24 horas antes del vencimiento, seguidos de escalaciones gerenciales en los días 1, 3 y 7 de retraso.
- **Auditoría y Análisis Predictivo (IA):** Con un solo clic, los auditores pueden generar reportes exportables en Excel para auditar qué material está fuera de la planta y quién es el responsable. Adicionalmente, el ecosistema integra Inteligencia Artificial para el análisis histórico y la generación de pronósticos de demanda de activos.

En conclusión, ZF Halo no es solo una herramienta de rastreo; es una capa vital de protección e innovación diseñada para modernizar de forma fluida y salvaguardar las operaciones diarias de ZF, empoderando a los empleados mediante una experiencia móvil sin fricciones y garantizando un 100% de cumplimiento normativo.

Usuarios/Roles

Manual de Perfiles de Usuario y Orquestación del Flujo (ZF Halo)

La arquitectura de ZF Halo se fundamenta en un ecosistema de Control de Acceso Basado en Roles (RBAC). El objetivo principal de la plataforma es la erradicación de procesos manuales y documentación en papel, digitalizando el ciclo de vida completo de los activos a través de interfaces optimizadas (Frontend) y validaciones transaccionales estrictas (Backend).

A continuación, se detalla la matriz de los 7 perfiles operativos del sistema:

1. Usuario General (Requisitor)

- **Contexto Operativo:** Ingenieros o colaboradores que requieren la asignación o préstamo temporal de equipos (ej. laptops, motores, prototipos) fuera de las instalaciones.
- **Capacidades en el Sistema:** Acceso multiplataforma (Web/Móvil) para consultar el catálogo de activos disponibles. Generación ágil de solicitudes (diseñada para completarse en menos de 3 interacciones), definiendo el tipo de salida (Con retorno, Sin retorno o *Scrap*) y las fechas proyectadas.
- **Interacción Arquitectónica:** El Frontend provee una interfaz intuitiva de selección ágil. Al confirmar, el Backend genera un registro inicial en la base de datos bajo el estatus "Pendiente de firmas", a la espera de la resolución del motor de reglas. Al ser aprobada, el sistema emite un Pase de Salida Digital (Código QR).

2. Gerente o Líder de Disciplina (Propietario del Activo)

- **Contexto Operativo:** Responsable directo de la custodia, asignación y resguardo del inventario asignado a su área o unidad de negocio.
- **Capacidades en el Sistema:** Sustitución del flujo de autorizaciones por correo electrónico mediante un sistema de notificaciones *in-app*. Capacidad para auditar solicitudes entrantes y emitir un dictamen (Aprobar/Rechazar). Visualización de un panel de control con el estado y ubicación en tiempo real de los equipos bajo su cargo.
- **Interacción Arquitectónica:** El Frontend despliega un *Dashboard* de autorizaciones pendientes. Al ejecutar una acción, el Backend registra su "Firma Electrónica" lógica y transiciona el estado de la solicitud hacia la siguiente fase del flujo.

3. Coordinador de S&R (Shipping & Receiving)

- **Contexto Operativo:** Personal responsable de la logística externa, empaquetado y coordinación con proveedores de transporte (ej. DHL).
- **Capacidades en el Sistema:** Operación centralizada mediante un panel tipo "Torre de Control". Visualización exclusiva de solicitudes previamente autorizadas por la Gerencia que requieren traslado externo. Habilitación de flujos específicos, como la confirmación de soporte logístico para empaquetado de residuos tecnológicos (*Scrap*).
- **Interacción Arquitectónica:** El Frontend renderiza un *Data Grid* avanzado con capacidades de filtrado por estado logístico (Pendientes de envío, En tránsito).

4. EHS (Medio Ambiente, Salud y Seguridad)

- **Contexto Operativo:** Personal normativo encargado de auditar el cumplimiento legal, ambiental y de seguridad industrial.
- **Capacidades en el Sistema:** Participación condicional basada en reglas de negocio ("By-pass" automático en salidas estándar). Su intervención se requiere exclusivamente cuando una solicitud es clasificada como *Scrap*, para auditar el riesgo ambiental y liberar legalmente la salida del equipo.
- **Interacción Arquitectónica:** El Frontend expone una vista dedicada y altamente filtrada que muestra únicamente las solicitudes de alto riesgo normativo pendientes de liberación.

5. Seguridad (Control de Accesos en Caseta)

- **Contexto Operativo:** Personal de vigilancia física en los puntos de control y aduanas perimetrales de la instalación.
- **Capacidades en el Sistema:** Operación mediante dispositivos móviles o tabletas en "Modo Kiosco". Escaneo ágil de Códigos QR (Pases de Salida) presentados por colaboradores o transportistas para validar instantáneamente la integridad de las firmas requeridas.
- **Interacción Arquitectónica:** El Frontend maximiza el área de escaneo de la cámara. Al procesar el QR, el Backend valida la cadena de firmas y, mediante la confirmación del guardia, registra un *timestamp* (marca de tiempo) exacto, actualizando el estatus del activo a "Fuera de planta" o "Devuelto".

6. Administrador Patrimonial (Superusuario)

- **Contexto Operativo:** Administrador global del ecosistema. No participa en el flujo transaccional diario, pero dicta los parámetros de operación.
- **Capacidades en el Sistema:** Gestión integral de los catálogos maestros (Usuarios, Activos, Categorías). Configuración del motor de reglas dinámicas de aprobación (Ej. *Regla A: Equipo de cómputo requiere 1 firma gerencial; Regla B: Maquinaria pesada requiere firma gerencial + validación S&R*).
- **Interacción Arquitectónica:** Consumo de módulos CRUD (Crear, Leer, Actualizar, Borrar) para la parametrización de la base de datos y programación de estatus de mantenimiento preventivo.

7. Auditor (Cumplimiento y Analítica)

- **Contexto Operativo:** Personal de finanzas, calidad o contraloría enfocado en la revisión de métricas y prevención de pérdidas patrimoniales.
- **Capacidades en el Sistema:** Acceso global con privilegios estrictos de solo lectura (*Read-Only*). Visualización de trazabilidad end-to-end y capacidad de exportación de reportes tabulares y *Dashboards* analíticos (PDF/Excel).

Orquestación del Flujo Logístico (Resumen Transaccional)

El ciclo de vida de una solicitud de salida sigue una automatización estricta impulsada por el backend:

1. **Creación:** El Usuario Requisitor genera la solicitud en la plataforma.
2. **Evaluación de Reglas:** El backend intercepta la solicitud y calcula dinámicamente las firmas requeridas según la configuración del Administrador Patrimonial.
3. **Autorización Primaria:** El Gerente de la disciplina ejecuta la Firma Electrónica 1.
4. **Validación Normativa (Condicional):** Si la solicitud es clasificada como *Scrap*, el perfil EHS ejecuta la Firma Electrónica 2.
5. **Validación Logística:** El Coordinador de S&R audita el transporte y ejecuta la Firma Electrónica 3 (Final).
6. **Emisión de Credencial:** El sistema consolida las aprobaciones y emite el Código QR encriptado al Requisitor.
7. **Cierre de Ciclo:** Seguridad escanea el Código QR en caseta, registrando la salida física del inventario de forma inmutable.

Requerimientos

1. Roles y Usuarios del Sistema

- **Usuario General:** Personal que requiere la asignación o préstamo de un equipo. Sus capacidades incluyen: consultar el catálogo de activos disponibles; levantar solicitudes de salida mediante un flujo optimizado (máximo 3 interacciones); definir parámetros logísticos (asignación con retorno, sin retorno o baja/scrap) junto con fechas estimadas; generar pases digitales mediante códigos QR tras la autorización del sistema; y reportar incidencias operativas al momento de la devolución.
- **Gerente o Líder de Disciplina:** Responsable directo de la gestión y autorización de equipos específicos. Sus funciones son: aprobar o rechazar solicitudes de salida mediante el sistema de notificaciones y visualizar en tiempo real la trazabilidad y asignación de los activos bajo su jurisdicción.
- **Coordinador de S&R (Shipping & Receiving):** Personal encargado de la logística externa. Sus responsabilidades abarcan: monitorear los envíos y salidas previamente aprobadas por la gerencia; validar la programación de transportistas o proveedores; y emitir la autorización logística para las salidas catalogadas como chatarra electrónica (Scrap).
- **EHS (Seguridad, Salud y Medio Ambiente):** Perfil responsable de la validación normativa. Su función principal es aprobar las evaluaciones de riesgo exclusivamente para salidas de tipo "Scrap", garantizando el cumplimiento de la normativa ambiental en el manejo de residuos.
- **Seguridad (Guardias en Caseta):** Personal encargado del control de accesos físicos. Operan el sistema mediante una interfaz móvil tipo Kiosco para ejecutar el *Check-out* y *Check-in* automático mediante el escaneo de códigos QR. El sistema valida en tiempo real las firmas electrónicas requeridas y registra la bitácora exacta de accesos.
- **Administrador Patrimonial:** Usuario con privilegios de configuración global. Sus capacidades incluyen: control total del catálogo (altas, bajas y modificaciones de activos, marcas, categorías y ubicaciones); configuración de reglas de negocio y flujos de aprobación dinámicos según la categoría del activo; y programación de mantenimientos preventivos con actualización automática de estatus.
- **Auditor:** Perfil enfocado en la revisión de métricas y cumplimiento. Cuenta con privilegios de solo lectura para auditar la trazabilidad completa, el historial de movimientos, los tiempos de respuesta y las aprobaciones del sistema.

2. Reglas de Negocio y Requerimientos Funcionales

El desarrollo backend debe apegarse estrictamente a las siguientes normativas lógicas:

- **Regla de Activos (Catálogos):** Todo registro de activo debe incluir obligatoriamente: identificador único, marca, modelo, número de serie, número de parte, descripción, categoría, estado operativo, ubicación física y un enlace para la generación de su código QR.
- **Regla de Destinos (Catálogos):** El sistema debe catalogar entidades o ubicaciones externas, registrando la razón social, datos de contacto y la relación de los activos asignados a dichas ubicaciones.
- **Regla de Autorización Dinámica (Flujo Logístico):** Las solicitudes de salida permanecerán restringidas y no visibles para el módulo de Seguridad (Caseta) hasta que el 100% de las firmas electrónicas requeridas alcancen el estatus de "Aprobado".
- **Regla de Transición por Incidencia (Flujo Logístico):** El reporte de daños o fallas durante el proceso de devolución desencadenará el cambio automático e inmediato del estado del activo a "En mantenimiento".
- **Regla de Recordatorios (Alertas):** El sistema ejecutará procesos automatizados para emitir notificaciones 48 y 24 horas previas a la fecha de vencimiento de un préstamo.
- **Regla de Escalación (Alertas):** Los retrasos en devoluciones activarán protocolos automáticos de escalación en los días 1, 3 y 7 de demora.
- **Regla de Visibilidad de Retrasos (Alertas):** Las notificaciones de escalación se distribuirán simultáneamente al usuario solicitante, a su gerente directo, al Administrador Patrimonial y al Director de la disciplina correspondiente.
- **Regla de Carga (Rendimiento):** El tiempo de respuesta para las transacciones de la API y el renderizado inicial de interfaces en el cliente no debe superar los 2 segundos.
- **Regla de Volumen (Rendimiento):** La arquitectura de la base de datos relacional debe garantizar la estabilidad operativa bajo simulaciones de carga de 10,000 registros de activos y 100 usuarios concurrentes.
- **Regla de Ciberseguridad:** Se implementarán medidas defensivas alineadas al estándar OWASP Top 10, exigiendo validación y sanitización estricta de parámetros de entrada para prevenir ataques de inyección SQL y Cross-Site Scripting (XSS).
- **Regla de Visualización (Dashboards):** La interfaz gráfica consumirá los servicios de la API para desplegar indicadores clave de rendimiento (KPIs), incluyendo: disponibilidad de inventario, top 10 de préstamos vencidos, métricas de demanda y distribución por unidad de negocio.
- **Regla de Exportación (Dashboards):** El sistema integrará la generación y descarga de reportes de trazabilidad e incidencias en formatos estandarizados (PDF y Excel).
- **Regla de Inteligencia Artificial:** Integración de un módulo analítico que procese el histórico de transacciones transaccional para emitir proyecciones predictivas de demanda de activos y automatizar reportes de comportamiento logístico.

3. Requerimientos No Funcionales

- **Tiempo de respuesta (Performance):** Las peticiones a la base de datos y la carga de vistas esenciales deben resolverse dentro de un umbral máximo de 2 segundos.
- **Tolerancia al volumen (Performance):** La infraestructura técnica soportará un nivel de concurrencia operativo de 100 usuarios y un volumen de almacenamiento de 10,000 registros sin degradación del servicio.
- **Separación de responsabilidades (Arquitectura):** Implementación del patrón cliente-servidor, desacoplando el sistema mediante un backend estructurado como API RESTful y un frontend desarrollado como Single Page Application (SPA).
- **Esquema de BD justificado (Arquitectura):** Diseño normalizado de base de datos relacional, sustentado técnicamente por un Modelo Entidad-Relación (DER) para garantizar la integridad referencial.
- **Control de versiones (Arquitectura):** Gestión del código fuente mediante el sistema Git, manteniendo un historial de confirmaciones (commits) estructurado, semántico y profesional.
- **Calidad de código (Arquitectura):** El código fuente deberá contar con documentación interna pertinente y apearse a las convenciones y estándares de codificación del ecosistema utilizado.
- **Autenticación y Autorización (Ciberseguridad):** Integración de un control de acceso basado en roles (RBAC) para la protección de rutas y recursos a nivel de API.
- **Prevención de vulnerabilidades (Ciberseguridad):** Mitigación proactiva de riesgos de seguridad, aplicando estrictamente el filtrado y validación de datos para neutralizar inyecciones de código y ataques XSS.
- **Diseño Responsivo (UX/UI):** Interfaz adaptable e integralmente operativa tanto en entornos de escritorio como en dispositivos móviles.
- **Eficiencia operativa (UX/UI):** La arquitectura de la información debe garantizar que el flujo principal de solicitud de activos se complete en un máximo de tres interacciones desde la vista principal.
- **Accesibilidad (UX/UI):** Cumplimiento de directrices de accesibilidad en el diseño de interfaces, asegurando contrastes normativos, legibilidad tipográfica y una jerarquía visual clara.

Tecnologías y Arquitectura

Stack Tecnológico

Stack Tecnológico y Arquitectura de Software

Para garantizar el cumplimiento de los Acuerdos de Nivel de Servicio (SLA) del proyecto específicamente tiempos de carga inferiores a 2 segundos bajo un estrés de 10,000 registros y 100 usuarios concurrentes, la plataforma ZF Halo ha sido construida sobre un *stack* tecnológico moderno, escalable y orientado a microservicios lógicos.

Stack de Tecnologías

- **Frontend:** React.js.
- **Build Tool:** Vite (Optimización de *bundling* y tiempos de compilación).
- **Estilos y UI:** Tailwind CSS (Enfoque *Mobile-First* y cumplimiento de accesibilidad A11y).
- **Backend:** Node.js con el framework Express.js.
- **Capa de Datos (ORM):** Prisma (Gestión de esquemas y migraciones).
- **Base de Datos:** PostgreSQL alojada en Supabase.
- **Control de Versiones y CI/CD:** Git y Docker.

Justificación del Motor de Base de Datos

¿Por qué SQL y no NoSQL?

El ecosistema de control patrimonial requiere una validación transaccional estricta. A diferencia de las bases de datos NoSQL (basadas en documentos), que priorizan la flexibilidad de esquemas, este sistema exige **integridad referencial absoluta**. Las operaciones involucran relaciones complejas (ej. un préstamo vincula a un Usuario, un Activo, múltiples Firmas Electrónicas y un Centro de Costos). Una base de datos relacional (SQL) garantiza el cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad), asegurando que no existan activos huérfanos ni firmas desvinculadas de su solicitud de origen.

¿Por qué PostgreSQL a través de Supabase?

PostgreSQL fue seleccionado por ser el Sistema de Gestión de Bases de Datos Relacionales (RDBMS) *open-source* más avanzado y robusto del mercado. Sus beneficios clave para este proyecto incluyen:

1. **Connection Pooling Nativo:** A través de la infraestructura de Supabase, se gestionan eficientemente las conexiones concurrentes de Prisma, evitando la saturación del servidor (*Timeouts*) al procesar las transacciones de los 10,000 activos.
2. **Trazabilidad en Tiempo Real:** Al estar alojado en la nube, centraliza la fuente de la verdad para todos los módulos (Frontend y Kiosco), eliminando inconsistencias locales.
3. **Escalabilidad:** Soporta consultas complejas y uniones (*Joins*) masivas sin degradación del tiempo de respuesta.

Patrones de Arquitectura

Single Page Application (SPA) y Progressive Web App (PWA)

El Frontend está diseñado bajo el patrón SPA. A diferencia de la web tradicional que recarga el Document Object Model (DOM) completo en cada interacción, la SPA renderiza dinámicamente solo los componentes necesarios mediante *Client-Side Routing* (React Router). Esto elimina la latencia visual, ofreciendo tiempos de respuesta instantáneos.

Adicionalmente, el sistema cuenta con capacidades PWA, permitiendo:

- Instalación directa en dispositivos móviles como una aplicación nativa.
- Operación eficiente en áreas de la planta con conectividad limitada mediante estrategias de almacenamiento en caché.
- Eliminación de dependencias de almacenamiento local, operando al 100% en la nube.

Arquitectura API RESTful

El Backend opera como un servidor de recursos independiente, completamente desacoplado de la vista.

- **Comunicación:** El intercambio de datos se realiza exclusivamente mediante cargas útiles (*payloads*) en formato JSON.
- **Seguridad (Middlewares):** Los *endpoints* están protegidos por un Control de Acceso Basado en Roles (RBAC). El sistema valida el token de autenticación (JWT) antes de procesar cualquier transacción, garantizando que usuarios generales no puedan ejecutar funciones de gerencia o auditoría.

Estructura de Directorios (Monorepo Lógico)

Para mantener la escalabilidad y separación de responsabilidades, el repositorio se organiza de la siguiente manera:

zf-halo-imu/

- └─ backend/ # API REST (Node.js + Prisma)
 - | └─ prisma/ # Definición del DER (schema.prisma)
 - | └─ src/
 - | └─ controllers/ # Lógica de negocio (Préstamos, firmas)
 - | └─ middleware/ # Seguridad (Validación JWT, OWASP)
 - | └─ routes/ # Definición de Endpoints
 - | └─ services/ # Notificaciones, Escalaciones e IA
 - | └─ scripts/ # Script de inyección masiva (10k activos)
 - | └─ Dockerfile
- └─ frontend/ # SPA (React + Vite)
 - | └─ src/
 - | └─ components/ # UI responsiva y componentes modulares
 - | └─ services/ # Clientes Axios para consumo de la API
 - | └─ views/ # Pantallas principales (Dashboard, Kiosco)
 - | └─ Dockerfile
- └─ docs/ # Documentación técnica (SAD, DER, IA Logs)
- └─ docker-compose.yml # Orquestador de contenedores
- └─ README.md # Documentación de despliegue

DevOps y Entorno de Desarrollo

Contenerización (Docker)

Para garantizar el principio de paridad entre los entornos de desarrollo, pruebas y producción, el sistema está completamente contenerizado. docker-compose orquesta tres servicios aislados:

1. **zf_web**: Contenedor del Frontend expuesto en el puerto 5173.
2. **zf_api**: Contenedor del Backend expuesto en el puerto 3000.
3. **zf_db**: Contenedor del motor PostgreSQL expuesto en el puerto 5432, con persistencia de datos asegurada mediante volúmenes virtuales.

Estrategia de Control de Versiones (Git Workflow)

Se adoptó un modelo de ramificación estandarizado para mitigar conflictos de integración:

- **main**: Rama exclusiva para código estable y verificado.
- **dev**: Rama base de integración continua.
- **Ramas de Trabajo**: Nombradas bajo prefijos funcionales (feat/, fix/) derivadas de dev. La integración se realiza exclusivamente mediante *Pull Requests*.

Estándar de Confirmaciones (Conventional Commits)

El equipo se rige por un estándar estricto de mensajería para mantener un historial semántico y automatizable. Cada *commit* debe ser atómico y utilizar imperativo presente:

Tipo	Propósito	Ejemplo Correcto
feat:	Nueva funcionalidad	feat: integra endpoint de usuarios
fix:	Corrección de un error (<i>bug</i>)	fix: resuelve crash al escanear QR
docs:	Cambios en documentación	docs: actualiza instrucciones de instalación
refactor:	Mejora de código sin cambiar lógica	refactor: optimiza consulta de activos
chore:	Mantenimiento o dependencias	chore: prepara build para producción

Políticas de Integración Continua

Para proteger la estabilidad del repositorio, se establecieron las siguientes políticas de confirmación (*Commit Rules*):

- **Criterio de Atomicidad:** Las confirmaciones deben representar una sola unidad lógica de trabajo. Se prohíbe el empaquetado de múltiples funciones inconexas en un solo *commit*.
- **Protección de Compilación:** Queda estrictamente prohibido realizar envíos (*push*) de código que rompa el flujo de ejecución (*Breaking builds*) hacia las ramas principales (dev o main).
- **Higiene del Repositorio:** Obligatoriedad del uso de *.gitignore* para excluir módulos de Node, archivos de entorno (*.env*) y registros de depuración locales.

Base de datos

GRUPO 1: Catálogos del Sistema (Listas Desplegables)

Descripción general: Tablas de búsqueda para evitar errores de escritura y mantener la integridad de los datos.

1. roles (Control de Accesos)

Descripción: Define los permisos de los usuarios en la app.

- id_rol (SERIAL PRIMARY KEY)
- nombre (VARCHAR UNIQUE NOT NULL)

2. disciplinas_areas (Departamentos - Viene de: Área)

Descripción: Los departamentos de ZF

- id_disciplina (SERIAL PRIMARY KEY)
- nombre (VARCHAR UNIQUE NOT NULL)

3. ubicaciones_fisicas (Lugares de almacenamiento interno)

Descripción: Estantes o laboratorios donde duerme el equipo.

- id_ubicacion (SERIAL PRIMARY KEY)
- nombre (VARCHAR UNIQUE NOT NULL)

4. categorias_activos (Clasificación - Viene de: Categoría)

Descripción: Clasificación del equipo.

- id_categoria (SERIAL PRIMARY KEY)
- nombre (VARCHAR UNIQUE NOT NULL)

5. estados_maquina (Salud del equipo - Viene de: Estado Máquina)

Descripción: Salud actual del activo.

- id_estado_maquina (SERIAL PRIMARY KEY)
- nombre (VARCHAR UNIQUE NOT NULL)

6. tipos_compra (Finanzas - Viene de: Tipo de compra)

Descripción: Catálogo financiero.

- id_tipo_compra (SERIAL PRIMARY KEY)
- nombre (VARCHAR UNIQUE NOT NULL)

7. tipos_nacionalidad (Aduanas - Viene de: Tipo nacional)

Descripción: Catálogo aduanal.

- id_tipo_nacionalidad (SERIAL PRIMARY KEY)
- nombre (VARCHAR UNIQUE NOT NULL)

8. destinos_externos (Universidades y Aliados para préstamos)

Descripción: Instituciones, universidades o proveedores a donde viaja el equipo.

- id_destino (SERIAL PRIMARY KEY)
- nombre_institucion (VARCHAR NOT NULL)
- tipo (VARCHAR NOT NULL) - Ej: 'Universidad', 'Proveedor'.
- contacto_nombre (VARCHAR NOT NULL)
- contacto_email (VARCHAR NULL)
- contacto_telefono (VARCHAR NULL)

9. centros_costo_proyectos (Finanzas - Viene de: Nombre del proyecto)

Descripción: Proyectos que pagaron el equipo.

- id_proyecto (SERIAL PRIMARY KEY)
- nombre (VARCHAR UNIQUE NOT NULL)

GRUPO 2: Personal y Seguridad

Descripción: Maneja las credenciales, el login corporativo y las jerarquías de aprobación.

10. usuarios

- id_usuario (SERIAL PRIMARY KEY)
- nombre_completo (VARCHAR NOT NULL)
- email (VARCHAR UNIQUE NOT NULL)
- password_hash (VARCHAR NOT NULL) - Contraseña encriptada.
- id_rol (INT NOT NULL) - Foreign Key -> roles
- id_disciplina (INT NOT NULL) - Foreign Key -> disciplinas_areas
- id_suplente (INT NULL) - Foreign Key -> usuarios (Para delegar firmas en vacaciones).
- secreto_2fa (VARCHAR NULL) - Llave de Google Authenticator.
- activo (BOOLEAN DEFAULT TRUE) - Baja lógica.

GRUPO 3: Inventario Central

Descripción: La mega-tabla que unifica los históricos del Excel ADAS 500 con los requerimientos modernos del código QR y los gerentes.

11. activos

- id_activo (SERIAL PRIMARY KEY)
- qr_codigo (VARCHAR UNIQUE NOT NULL) - *Hash para la tablet del guardia.*
- numero_serie (VARCHAR UNIQUE NOT NULL)
- numero_parte (VARCHAR NULL)
- marca (VARCHAR NOT NULL)
- modelo (VARCHAR NOT NULL)
- anio (INT NULL)
- nombre_maquina (VARCHAR NOT NULL)
- tag (VARCHAR NULL) - *Ej: 'MNR-ADAS-00001'*
- subarea (VARCHAR NULL) - *Ej: 'A0001'*
- descripcion (TEXT NULL)
- comentarios (TEXT NULL)
- cantidad_inicial (DECIMAL NULL) - *Solo para categoría Refacciones.*
- cantidad_actual (DECIMAL NULL) - *Solo para categoría Refacciones.*
- pedimento (VARCHAR NULL) - *Aduanas.*
- factura (VARCHAR NULL)
- valor_comercial (DECIMAL NULL)
- fecha_compra (DATE NULL)
- es_compra (BOOLEAN NULL)
- **Llaves Foráneas (Obligatorias):**
 - id_disciplina (INT NOT NULL) -> disciplinas_areas
 - id_categoria (INT NOT NULL) -> categorias_activos
 - id_estado_maquina (INT NOT NULL) -> estados_maquina
 - id_ubicacion (INT NOT NULL) -> ubicaciones_fisicas
 - id_gerente_responsable (INT NOT NULL) -> usuarios (El dueño que aprueba).
- **Llaves Foráneas (Opcionales, según Excel):**
 - id_proyecto (INT NULL) -> centros_costo_proyectos
 - id_tipo_compra (INT NULL) -> tipos_compra
 - id_tipo_nacionalidad (INT NULL) -> tipos_nacionalidad

GRUPO 4: Flujo, Burocracia y Auditoría

Descripción: El motor transaccional de la app. Gestiona intenciones de viaje, firmas y los historiales de infracciones para los dashboards.

12. solicitudes (El carrito de peticiones)

Descripción: Registra la petición de viaje del equipo.

- id_solicitud (SERIAL PRIMARY KEY)
- id_activo (INT NOT NULL) -> activos
- id_usuario_solicitante (INT NOT NULL) -> usuarios
- id_destino (INT NULL) -> destinos_externos
- tipo_salida (VARCHAR NOT NULL) - Ej: 'Retorno', 'Scrap'.
- estatus_general (VARCHAR NOT NULL) - Ej: 'Pendiente', 'Aprobada'.
- fecha_creacion (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- fecha_salida_programada (TIMESTAMP NULL)
- fecha_devolucion_programada (TIMESTAMP NULL)
- metodo_transporte (VARCHAR NOT NULL) - Ej. "Paquetería DHL", "Recolección por Proveedor"

13. aprobaciones_firma (Firmas individuales)

Descripción: Separa las firmas para que múltiples jefes aprueben una sola solicitud independientemente.

- id_firma (SERIAL PRIMARY KEY)
- id_solicitud (INT NOT NULL) -> solicitudes
- id_rol_esperado (INT NOT NULL) -> roles (A quién le toca).
- id_usuario_firmo (INT NULL) -> usuarios (Quién lo hizo).
- estatus_firma (VARCHAR NOT NULL) - Ej: 'Pendiente', 'Aprobada'.
- comentarios (TEXT NULL)
- fecha_firma (TIMESTAMP NULL)

14. reglas_aprobacion (Panel de control del Admin)

Descripción: Configuración del administrador para decidir qué firmas exige cada tipo de activo.

- id_regla (SERIAL PRIMARY KEY)
- id_categoria (INT UNIQUE NOT NULL) -> categorias_activos
- requiere_gerente (BOOLEAN DEFAULT TRUE)
- requiere_syr (BOOLEAN DEFAULT FALSE)
- requiere_ehs (BOOLEAN DEFAULT FALSE)

15. registro_alertas_retrasos

Descripción: El historial de infracciones para que el Auditor grafique KPIs y reportes.

- id_alerta (SERIAL PRIMARY KEY)
- id_solicitud (INT NOT NULL) -> solicitudes (El préstamo que se venció).
- id_usuario_infractor (INT NOT NULL) -> usuarios (Quién tiene el equipo).
- tipo_alerta (VARCHAR NOT NULL) - Ej: 'Recordatorio 24h', 'Escalación Día 3'.
- fecha_envio (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

GRUPO 5: Operación Física

Descripción: Registro de seguridad física en puertas y reparaciones técnicas.

16. bitacora_caseta (Caja negra de los guardias)

Descripción: Registro inmutable de cuándo escanean el QR en la puerta.

- id_bitacora (SERIAL PRIMARY KEY)
- id_solicitud (INT NOT NULL) -> solicitudes
- id_guardia (INT NOT NULL) -> usuarios
- tipo_movimiento (VARCHAR NOT NULL) - 'Salida' o 'Retorno'.
- fecha_hora_real (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

17. mantenimientos_incidentes

Descripción: Reportes de daños al devolver equipos o programaciones a futuro.

- id_mantenimiento (SERIAL PRIMARY KEY)
- id_activo (INT NOT NULL) -> activos
- id_reportador (INT NOT NULL) -> usuarios
- id_solicitud_origen (INT NULL) -> solicitudes (Si se rompió durante un préstamo).
- tipo_mantenimiento (VARCHAR NOT NULL) - 'Preventivo' o 'Correctivo'.
- descripcion (TEXT NOT NULL)
- estatus_reparacion (VARCHAR NOT NULL) - 'Abierto', 'Cerrado'.
- fecha_reporte (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- fecha_programada (TIMESTAMP NULL) - Para agendar a futuro.
- fecha_resolucion (TIMESTAMP NULL)
- comentarios_resolucion (TEXT)

Endpoints

Método y Endpoint	¿Qué hace este endpoint?	Roles con Acceso Exacto	React Envía (Body)	Node.js Responde
POST /api/auth/login	Verifica credenciales y reCAPTCHA. Nota: En el actual <i>Modo Desarrollo</i> omite el 2FA y entrega la sesión directa. En <i>Modo Producción</i> validará si es necesario mostrar el QR o solo pedir el código.	Administrador, Gerente, Usuario, S&R, EHS, Seguridad, Auditor	{ email, password, captchaToken }	(Desarrollo): { token, user: { id, nombre, id_rol } } (Producción): { requires2fa: true, userId, setupRequired: boolean }
POST /api/auth/setup-2fa	NUEVO. Genera la llave maestra 2FA para la app de autenticación, la guarda en la base de datos y devuelve la	Administrador, Gerente, Usuario, S&R, EHS, Seguridad, Auditor	{ userId }	{ mensaje, qrImage, secretoManual }

	imagen del código QR.			
POST /api/auth/verify-2fa	Valida el PIN temporal (con 30s de margen) contra el secreto guardado. Si coincide, entrega la "Llave Electrónica" (JWT).	Administrador, Gerente, Usuario, S&R, EHS, Seguridad, Auditor	{ userId, codigo2FA }	{ token, user: { id, nombre, id_rol } }
GET /api/auth/me	Extrae el ID del token JWT del Header y devuelve la información actualizada del perfil desde la base de datos.	Administrador, Gerente, Usuario, S&R, EHS, Seguridad, Auditor	(Ninguno, lee el Header)	{ id_usuario, nombre_completo, id_rol, id_disciplina }

Módulo 2: Catálogos y Destinos Externos

Método y Endpoint	¿Qué hace este endpoint?	Roles con Acceso	React Envía (Body/Params)	Node.js Responde
GET /api/catalogos/:tipo	Endpoint dinámico que devuelve listas de: disciplinas, ubicaciones, categorías, estados, roles, etc.	Todos los roles	params: { tipo }	[{ id, nombre }, ...]
POST /api/catalogos/destinos_externos	Registra una nueva institución (ej. Universidad o Taller) para salidas de activos.	Admin, Gerente	{ nombre_institucion, tipo, contacto_nombre, ... }	{ status: "success", new_id_destino }
PUT /api/catalogos/destinos_externos/:id	Actualiza la información de contacto de un destino externo existente.	Admin, Gerente	params: { id }, body: { contacto_nombre, ... }	{ status: "success", destino_actualizado: true }

Módulo 2: Inventario de Activos (Core)

Método y Endpoint	¿Qué hace este endpoint?	Roles con Acceso	React Envía (Query/Body)	Node.js Responde
GET /api/activos	Lista activos con paginación. Permite filtrar por categoría, estado o búsqueda (Q). Soporta ordenamiento (A-Z, Recientes, etc).	Todos los roles	query: { page, limit, q, orden, id_categoria }	{ data: [...], meta: { total, totalPaginas } }
GET /api/activos/:id	Busca un activo específico por su ID numérico o escaneando su Código QR .	Todos los roles	params: { id } (puede ser ID o UUID del QR)	{ id_activo, nombre_maquina, categoria, ubicacion, ... }
POST /api/activos	Registra un activo. Si no se envía QR, el sistema genera uno automáticamente (UUID).	Admin, Gerente	{ nombre_maquina, numero_serie, id_categoria, ... }	{ status: "success", activo: { ... } }

PUT /api/activos/:id	Modifica los datos técnica o asignación de un activo.	Admin, Gerente	body: { ...datos a actualizar }	{ status: "success", data: { activo_actualizado } }
DELETE /api/activos/:id	"Baja" lógica del activo. No se borra, cambia su estado a "Baja" (ID: 4).	Admin, Gerente	params: { id }	{ status: "success", mensaje: "Activo dado de baja" }
GET /api/activos/:id/trazabilidad	HISTORIAL COMPLETO. Genera una línea de tiempo unificando: Alta, Solicitudes (Préstamos) y Mantenimientos.	Admin, Gerente, Auditor	params: { id }	{ id_activo, nombre, historial: [{ tipo_evento, fecha, descripcion }] }

Módulo 3: Flujo, Burocracia y Firmas (El Corazón del Reto)

Método y Endpoint	¿Qué hace este endpoint?	Roles con Acceso	React Envía	Node.js Responde

POST /api/solicitudes	Lógica Bimodal: 1. Si es Admin : Crea asignación directa (Estatus: Aprobada). 2. Si es Usuario : Crea solicitud y dispara cadena de firmas según la categoría del activo.	Todos (Protegido)	{ id_activo, tipo_salida, id_destino, ... }	{ id_solicitud, estatus_general, mensaje }
GET /api/solicitudes/master	Historial maestro de la planta. Los Gerentes solo ven su propia disciplina; Admins y Auditores ven todo.	Admin, Gerente, S&R, Auditor	query: { estatus, page }	{ data: [...], paginacion: { total } }
GET /api/solicitudes/mis	Bandeja personal del usuario para ver el avance de sus folios y firmas.	Todos	query: { q, estatus }	{ data: [{ folio, activo, estatus }] }

GET /api/solicitudes/:id	Detalle profundo de un folio, incluyendo el estatus individual de cada firma (Gerente, S&R, EHS).	Todos	params: { id }	{ id_solicitud, firmas: { gerente, syr, ehs }, ... }
PATCH /api/solicitudes/:id/cancelar	Permite al dueño o al Admin cancelar un folio si aún está en estado "Pendiente". Libera el activo automáticamente.	Dueño o Admin	params: { id }	{ status: "success", estatus_general: "Cancelada" }

Método y Endpoint	¿Qué hace este endpoint ?	Roles con Acceso	React Envía	Node.js Responde
GET /api/aprobaciones/pendientes	Muestra solo los folios que el usuario actual debe firmar. Incluye lógica de Suplentes (firmar por otro).	Gerente, S&R, EHS, Admin	(Ninguno)	[[id_firma, solicitud: { activo, solicitante }]]

PATCH /api/aprobaciones/dictaminar/:id_firma	Acción de Firmar: Registra "Aprobada" o "Rechazada". Si es la firma de S&R, cambia el activo a "Prestado" y el folio a "En Tránsito".	Gerente, S&R, EHS, Admin	{ estatus_firma, comentarios }	{ status: "success", estatus_general_solicitud }
GET /api/aprobaciones/logistica/historial	Panel especializado para Caseta/S&R. Organiza folios en pestañas: Pendientes de envío, En tránsito y Devueltos.	S&R (Logística), Admin	(Ninguno)	[{ folio, qr_codigo, estado: "En tránsito", ... }]

Módulo 4: Operación Física (Caseta y Mantenimiento)

Método y Endpoint	¿Qué hace este endpoint?	Roles con Acceso	React Envía (Body)	Node.js Responde
POST <code>/api/bitacora/checkout</code>	SALIDA: El guardia escanea el QR. El sistema verifica que el folio esté "Aprobado". Si es así, registra el movimiento y cambia el activo a "Prestada".	Seguridad, Admin	<code>{ qr_codigo }</code>	<code>{ status: "success", mensaje: "Salida autorizada" }</code>
POST <code>/api/bitacora/checkin</code>	ENTRADA: El guardia registra el retorno. Cierra el folio como "Devuelto" y libera la máquina cambiándola a "Operativa" (Disponible).	Seguridad, Admin	<code>{ qr_codigo }</code>	<code>{ status: "success", mensaje: "Equipo devuelto al almacén" }</code>
GET <code>/api/bitacora</code>	HISTORIAL: Lista de los últimos 100 movimientos de entrada/salida para auditoría rápida.	Seguridad, Auditor, Admin	<i>(Ninguno)</i>	<code>[{ id_bitacora, tipo_movimiento, guardia, activo, fecha_hora_real }]</code>

Método y Endpoint	¿Qué hace este endpoint?	Roles con Acceso	React Envía (Body)	Node.js Responde
GET /api/mantenimientos	Lista todos los reportes de daño levantados, indicando quién reportó y qué equipo está en el taller.	Todos los roles	(Ninguno)	[{ id_mantenimiento, descripcion, estatus_reparacion, activo: { ... } }]
POST /api/mantenimientos	REPORTE DE DAÑO: Crea un ticket y bloquea la máquina automáticamente (ID 2: En Mantenimiento). Nadie podrá pedirla prestada mientras esté así.	Todos (Protegido)	{ id_activo, tipo_mantenimiento, descripcion }	{ status: "success", mensaje: "Reporte creado y equipo bloqueado" }
PATCH /api/mantenimientos/:id	CIERRE/REPARACIÓN: El técnico marca el equipo como reparado. El sistema cambia el activo a "Operativa" (ID 1) para que vuelva a estar disponible.	Admin, Gerente, EHS	params: { id }	{ status: "success", activo_liberado: true }

Módulo 5: Administración de Personal (Usuarios y Suplentes)

Método y Endpoint	¿Qué hace este endpoint?	Roles con Acceso	React Envía	Node.js Responde
GET /api/usuarios	Lista de empleados para asignación de activos o firmas. Permite filtrar por activos/inactivos.	Admin, Gerente, S&R	query: { incluir_inactivos }	[[id_usuario, nombre_completo, rol, disciplina, suplente]]
GET /api/usuarios/:id	Expediente del Empleado: Muestra datos de contacto y el historial de sus últimas 5 solicitudes y 5 firmas realizadas.	Admin, Gerente, Auditor	params: { id }	{ id_usuario, ..., solicitudes_solicitante: [], firmas_realizadas: [] }
POST /api/usuarios	Alta de Personal: Registra un nuevo empleado con contraseña encriptada (bcrypt).	Solo Administrador	{ nombre_completo, email, password, id_rol, id_disciplina }	{ status: "success", id_usuario }

PATCH /api/usuarios/:id	Actualización / Suplencia: Modifica datos del usuario o asigna a otro empleado como su Suplente oficial.	Solo Administrador	{id_suplente, activo, id_rol, ... }	{ status: "success", mensaje: "Usuario actualizado" }
DELETE /api/usuarios/:id	Baja Lógica: Desactiva la cuenta del empleado. No se borra para no romper el historial de firmas pasadas.	Solo Administrador	params: { id }	{ status: "success", mensaje: "Usuario dado de baja" }

Concepto	Implementación en Backend	Impacto en el Sistema
Encriptación	Uso de <code>bcryptjs</code> con 10 rounds de sal.	Las contraseñas nunca se guardan en texto plano en la base de datos de ZF.
Baja Lógica	El campo <code>activo</code> pasa a <code>false</code> .	El usuario ya no puede loguearse, pero su nombre sigue apareciendo en los reportes de auditoría de folios antiguos.

Jerarquía de Firmas	Vinculación <code>id_rol</code> e <code>id_disciplina</code> .	Asegura que un Gerente de "Sistemas" solo reciba solicitudes de activos de su propia área.
Delegación (Suplente)	Relación self-referencing en la tabla <code>usuarios</code> .	Si el Gerente titular no está, el sistema permite que el ID del suplente sea validado en la bandeja de aprobaciones pendientes.

Módulo 6: Reglas de Negocio

Este módulo controla la tabla `reglas_aprobacion`.

Método y Endpoint	¿Qué hace este endpoint?	Roles con Acceso	React Envía (Body/Params)	Node.js Responde
GET <code>/api/reglas</code>	Obtiene el listado completo de categorías y sus requisitos de firma actuales (Gerente, S&R, EHS).	Admin, Auditor	(Ninguno)	<code>{ id_categoria, requiere_gerente, requiere_syr, ... }</code>
GET <code>/api/reglas/:id</code>	Consulta la configuración específica para una categoría (ej. "Maquinaria Pesada"). Si no existe, devuelve valores por defecto.	Admin, Gerente	<code>params: { id_categoria }</code>	<code>{ id_categoria, requiere_gerente: true, ... }</code>

PATCH <code>/api/reglas/:id</code>	CONFIGURACIÓN : Permite activar o desactivar firmas obligatorias para una categoría específica.	Solo Administrador	{ requiere_gerente, requiere_syr, requiere_ehs }	{ status: "success", regla: { ... } }
--	---	--------------------	---	---

Módulo 7: Dashboards, Alertas y Reportes (Auditoría)

Este módulo expone la información de la tabla `registro_alertas_retrasos` y cruza datos de `solicitudes` y `activos`.

Método y Endpoint	¿Qué hace este endpoint?	Roles con Acceso	React Envía	Node.js Responde
GET <code>/api/dashboard/kpis</code>	Calcula en tiempo real: equipos fuera, % de retrasos, infractores y destinos más comunes.	Admin, Gerente, Auditor	(Ninguno)	{ kpis: {...}, estatus: {...}, lista_vencidos: [] }

GET <code>/api/reportes/historial</code>	Visualización Web: Lista paginada de todos los movimientos históricos para consulta rápida en pantalla.	Admin, Gerente, Auditor	query: { page, limit }	{ metadata: {...}, data: [...] }
GET <code>/api/reportes/exportar</code>	Generación de Excel: Crea un archivo .xlsx profesional con filtros de fecha para auditorías externas.	Admin, S&R, Auditor	query: { fecha_inicio, fecha_fin }	Archivo Descargable (Buffer Excel)
GET <code>/api/trazabilidad/:id</code>	Timeline del Activo: Reconstruye toda la vida de un equipo: desde su compra, quién lo ha usado y cuántas veces se ha roto.	Admin, Auditor, S&R	params: { id }	{ activo: {...}, historial: [{tipo, detalle, fecha}] }

Módulo 8: Sistema de Notificaciones Inteligentes

Rol del Usuario	Tipo de Notificación	Fuente de Datos (Prisma)	Prioridad Visual

Admin / Auditor	Infracciones de toda la planta.	registro_alertas_retrasos	Alta (Rojo): Alertas de sistema por vencimientos.
Gerente	Firmas de aprobación pendientes.	aprobaciones_firma	Media (Azul): Peticiones de activos de su personal.
Usuario / Empleado	Estatus de sus trámites + Alertas personales.	solicitudes & registro_alertas	Variable: Éxito (Verde) para aprobaciones, Alerta (Rojo) para sus propios retrasos.